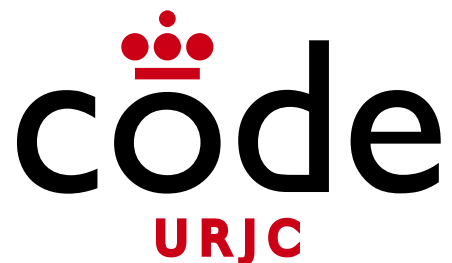


Fundamentos de la Web

Bloque III: Tecnologías de servidor web

Tema 3.4: Módulos en JavaScript (ESM)



©2024

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Módulos

- JavaScript permite dividir un programa en diferentes ficheros .js (**módulos**)
- Mejora la **mantenibilidad** del código
 - Abstracción y encapsulación
 - Comprensión
 - Reutilización
- El sistema de módulos estándar se denomina

ES Modules (EcmaScript Modules, ESM)

Módulos

- Cada **fichero** es un **módulo**
- Si un módulo A quiere usar variables o funciones de un módulo B:
 - El módulo B tiene que **exportar** lo que quiera hacer público

export

- El módulo A tiene que **importar** lo que quiera usar del módulo B

import ... from

Módulos

ejem1-import

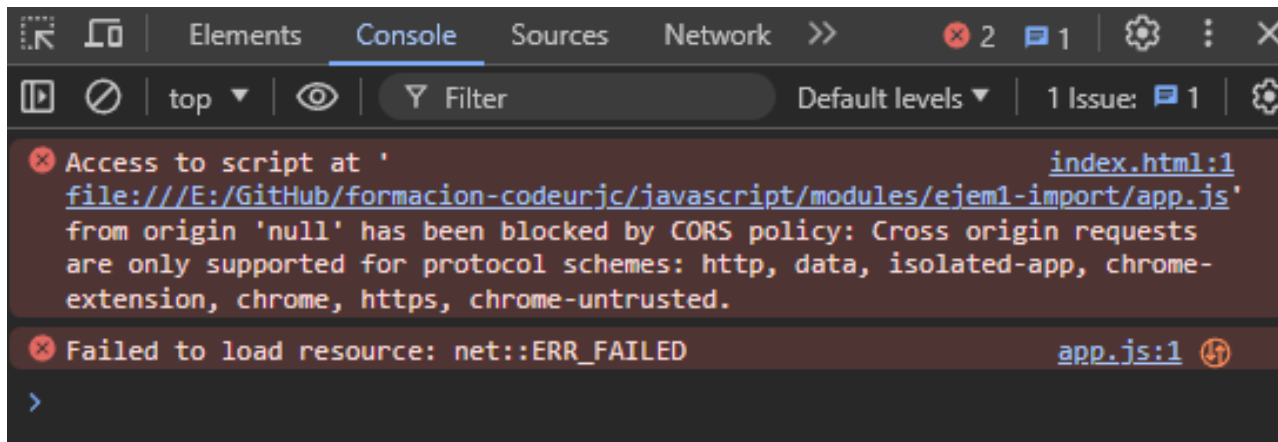
- Uso de módulos en el navegador:
 - 1) Añadir **type="module"** al tag **<script>**:

```
<html>
  <head>
    <script type="module" src="app.js"></script>
  </head>
  <body></body>
</html>
```

Indica al navegador que
el código JavaScript
es un módulo

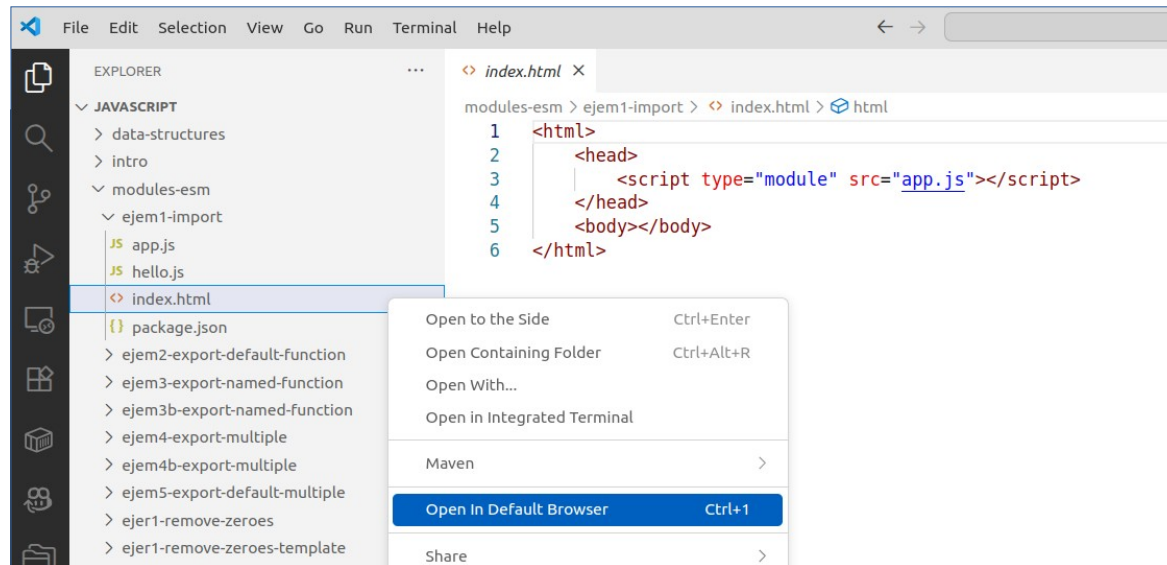
ejem1-import

- Uso de módulos en el navegador:
 - 2) Usar un **servidor web**
 - Si abrimos la web haciendo doble click en el fichero .html obtenemos un error (**file://**)



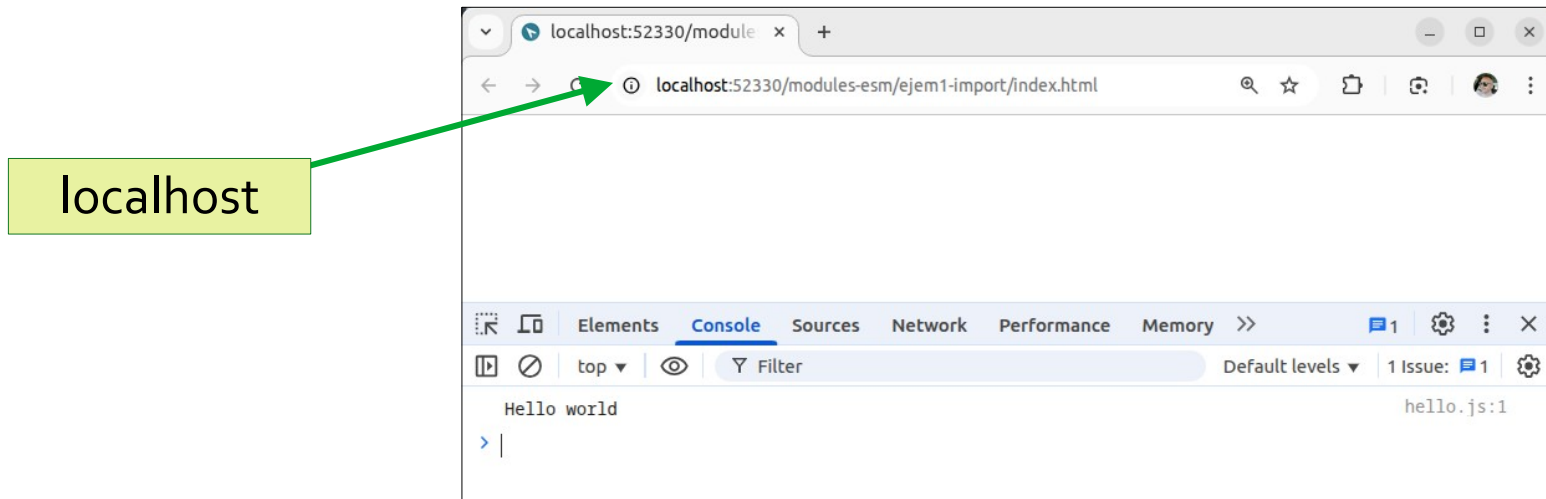
ejem1-import

- Uso de módulos en el navegador:
 - 2) Usar un **servidor web**
 - En desarrollo basta con usar la extensión de VSCode “Open in Default Browser” (http://)



ejem1-import

- Uso de módulos en el navegador:
 - 2) Usar un **servidor web**
 - En desarrollo basta con usar la extensión de VSCode “Open in Default Browser”



Módulos

`ejem1-import`

- Uso de módulos en el **navegador**:
 - 2) Usar un **servidor web**
 - Si estamos desarrollando una **aplicación web** (con servidor dinámico) los ficheros JavaScript serán accesibles por http / https y funcionará correctamente

Módulos

ejem1-import

- Uso de módulos en **Node.js**:
 - Existen diversas estrategias
 - Usaremos la configuración en el fichero `package.json`

`package.json`

```
{
  "type": "module"
}
```

```
• mica@mica-laptop:~/git/javascript/modules-esm/ejem1-import$ node app.js
Hello world
```

`ejem1-import`

- Mecanismos de exportación e importación
 - 1) Ejecutar código de un módulo
 - 2) Usar el único elemento exportado de un módulo
 - 3) Usar uno o varios elementos exportados de un módulo

- Mecanismos de exportación e importación
 - 1) Ejecutar código de un módulo

hello.js

```
console.log('Hello world');
```

app.js

```
import './hello.js';
```

Si el módulo importado tiene sentencias, se ejecutan al importarlo

- Mecanismos de exportación e importación
 - 2) Usar el único elemento exportado de un módulo
 - Se exporta con **default** y se le da nombre al importar

bar.js

```
export default function() {  
    console.log('bar!');  
}
```

app.js

```
import bar from './bar.js';  
  
bar();
```

- Mecanismos de exportación e importación
 - 3) Usar **uno** o varios elementos exportados de un módulo

fiz.js

```
export function fiz() {  
    console.log('fiz!');  
}
```

app.js

```
import { fiz } from './fiz.js';  
  
fiz();
```

- Mecanismos de exportación e importación
 - 3) Usar uno o **varios** elementos exportados de un módulo

utils.js

```
export let obj = {  
  prop1: 3,  
  prop2: "value"  
}  
  
export function log(msg) {  
  console.log(msg);  
}  
  
export class Date {  
  constructor(date) {  
    this.date = date;  
  }  
}
```

app.js

```
import { obj, log, Date } from './utils.js';  
  
log("message");  
  
console.log(obj);  
  
let date = new Date('01-12-19');
```

- Mecanismos de exportación e importación
 - 3) Usar uno o **varios** elementos exportados de un módulo

utils.js

```
export let obj = {  
  prop1: 3,  
  prop2: "value"  
}  
  
export function log(msg) {  
  console.log(msg);  
}  
  
export class Date {  
  constructor(date) {  
    this.date = date;  
  }  
}
```

app.js

```
import * as utils from './utils.js';  
  
utils.log("message");  
  
console.log(utils.obj);  
  
let date = new utils.Date('01-12-19');
```

Se pueden importar todos en
un único objeto

Ejercicio 1

- Mueve la función de quitar ceros en un módulo
- Exporta por defecto la función desde un módulo
- Haz que el fichero app.js use la función del módulo

